VTC 242 — Cahier des charges v3

Sep 30, 2026 · @Scott

Application web responsive de mobilité urbaine (République du Congo). Document destiné à l'équipe de développement.

1. Présentation et objectifs

VTC 242 met en relation passagers et chauffeurs (taxis conventionnés et véhicules particuliers) à Brazzaville, avec trois garanties : chauffeur identifié, course démarrée par un code de départ à 4 chiffres, paiement par wallet alimenté en Mobile Money (MTN MoMo, Airtel Money).

La V1 est une **application web responsive** (téléphone, tablette, ordinateur) installable comme PWA. Les applications Android et iOS viendront en V2 en réutilisant la même API. Le prototype cliquable livré en septembre 2026 (pages passager, chauffeur, back-office) fait office de référence UX : les écrans, libellés et couleurs décrits ici s'y rapportent.

Objectif	Mesure de succès V1
Lancer commercialement à Brazzaville	30 chauffeurs actifs et 200 courses/semaine à M+2 après mise en production
Sécuriser chaque course	100 % des courses démarrées avec code de départ validé ; alerte de déviation < 20 s
Paiement fiable	Taux de réussite des recharges Mobile Money ⩾ 95 % ; 0 écart de réconciliation
Fonctionner sur réseaux faibles	Application utilisable en 3G, poids initial < 500 Ko compressé

Comment lire ce document. Les sections 2 à 4 décrivent le produit (périmètre, parcours, fonctionnalités). Les sections 5 à 11 sont techniques (architecture, données, API, paiements, sécurité, hébergement). Les sections 12 à 14 cadrent le projet (planning, tests, risques). Les exigences sont numérotées (F-xx fonctionnelles, N-xx non fonctionnelles) pour être reprises dans le backlog.

2. Périmètre et versions

La V1 livre une application web complète pour trois rôles (passager, chauffeur, administrateur), une seule ville, le transport immédiat uniquement. Tout ce qui suit est conçu pour que les V2 et V3 s'ajoutent sans refonte de l'API ni du modèle de données.

Version	Contenu	Hors périmètre
V1 — Web responsive (ce document)	Inscription par SMS · commande immédiate (taxi, standard, VIP, mariage, utilitaire) · estimation de prix · dispatch automatique · code de départ · suivi GPS temps réel · détection de déviation · chat in-app · bouton d'urgence · notation · wallet + Mobile Money (recharge, réservation de fonds, paiement, retrait chauffeur) · espèces en option · back-office (supervision, validation chauffeurs, tarifs, zones, finances, litiges) · PWA installable	Réservation à l'avance, covoiturage, location, comptes entreprise, multilingue
V2 — Apps Android / iOS	React Native (Expo) consommant la même API · GPS en arrière-plan côté chauffeur · notifications push natives · publication sur les stores	—
V3 — Extensions	Réservation à l'avance · covoiturage · location avec/sans chauffeur · comptes entreprise et facturation · Pointe-Noire · lingala / kituba	—

Trois applications web dans un seul projet frontend. Le passager et le chauffeur utilisent la même application (routes `/p/*` et `/d/*`, interface mobile-first) ; l'administrateur utilise le back-office (`/admin/*`, interface desktop-first). Un même compte ne peut pas être à la fois passager et chauffeur en V1.

Contraintes V1 à connaître avant de commencer

- Un navigateur mobile ne remonte pas la position GPS quand l'onglet est en arrière-plan ou l'écran verrouillé. Côté chauffeur, l'application demande un *Wake Lock* (écran allumé) pendant qu'il est en ligne et affiche un bandeau si la position n'est plus reçue depuis 30 s. C'est la principale raison de la V2 native.
- Les notifications utilisent Web Push (fonctionne sur Android/Chrome ; sur iOS uniquement si la PWA est installée sur l'écran d'accueil, iOS 16.4+). Un SMS est envoyé en complément pour les événements critiques (chauffeur trouvé, code de départ).

3. Acteurs et parcours de référence

Acteur	Accès	Ce qu'il fait
Passager	`/p/*`, compte par numéro de téléphone	Commande, suit, paie, note, conteste
Chauffeur	`/d/*`, compte validé après contrôle des documents	Se met en ligne, accepte des offres, saisit le code, termine, retire ses gains
Administrateur	`/admin/*`, rôle `admin`	Tout le back-office, y compris tarifs et finances
Finance	`/admin/*`, rôle `finance`	Tarifs, retraits, exports, réconciliation
Support	`/admin/*`, rôle `support`	Supervision, chat, litiges, blocage ; aucun paramètre financier

Parcours nominal d'une course (à reproduire tel quel dans les tests d'acceptation)

1. Le passager saisit sa destination ; le point de départ est proposé par GPS et modifiable.
2. Il choisit une catégorie ; l'application affiche une fourchette de prix (± 10 %) et le temps d'approche estimé.
3. Il confirme. Le backend vérifie le solde du wallet et **réserve** le montant haut de la fourchette (ou accepte le mode espèces si activé pour la catégorie).
4. Le dispatch propose la course aux chauffeurs disponibles les plus proches, un par un, 15 s chacun.
5. À l'acceptation, le passager reçoit la fiche chauffeur (photo, prénom, note, marque, modèle, couleur, plaque, n° taxi) et son **code de départ à 4 chiffres**. Le suivi d'approche démarre.
6. Le chauffeur signale son arrivée ; l'attente gratuite (3 min par défaut) démarre.

7. Le passager donne le code oralement ; le chauffeur le saisit. Le backend le valide, fige l'itinéraire planifié et passe la course en cours.
8. Positions toutes les 3 à 5 s ; alerte de déviation confirmée après 3 positions consécutives à plus de 300 m de l'itinéraire.
9. Le chauffeur clôture à destination (bouton actif dans un rayon de 200 m ou validation du passager). Prix final calculé, montant réservé débité, commission prélevée, wallet chauffeur crédité.
10. Les deux parties se notent. Le passager dispose de 24 h pour contester.

Il n'existe pas de « code de fin de course » : la géolocalisation d'arrivée et la fenêtre de contestation remplissent ce rôle.

4. Exigences fonctionnelles

Chaque ligne est une exigence testable. Priorité : M = indispensable au lancement, S = souhaitable en V1, V2 = reportée.

4.1 Compte et authentification (communs)

Réf.	Exigence	Prio
F-01	Inscription et connexion par numéro +242 avec OTP SMS à 6 chiffres, valable 5 min, 3 essais, puis blocage 15 min	M
F-02	Session persistante 30 jours (refresh token rotatif) ; déconnexion à distance depuis le back-office	M
F-03	Profil : prénom, nom, photo facultative (passager) / obligatoire (chauffeur), langue (français seul en V1)	M
F-04	Suppression de compte : anonymisation des données personnelles, conservation des transactions 10 ans	M
F-05	Un seul rôle par compte (passager ou chauffeur) ; un chauffeur a un seul appareil actif	M

4.2 Passager (`/p/*`)

Réf.	Exigence	Prio
F-10	Écran d'accueil : carte, position GPS, taxis proches, champ « Où allez-vous ? », onglets Taxis / Voitures personnelles, bouton Urgence permanent	M
F-11	Recherche d'adresse : autocomplétion OSM (Photon) + référentiel de points d'intérêt locaux administrable ; saisie libre acceptée	M
F-12	Catégories : Taxi conventionné, Standard, VIP, Mariage & événements (sur devis), Utilitaire / déménagement	M
F-13	Estimation avant commande : fourchette de prix, distance, durée, temps d'approche ; recalcul si la destination change	M
F-14	Choix du paiement : wallet (défaut) ou espèces (si activé pour la catégorie et la zone)	M
F-15	Commande : réservation des fonds, écran « recherche d'un chauffeur », annulation gratuite tant qu'aucun chauffeur n'a accepté	M
F-16	Chauffeur trouvé : fiche complète + code de départ affiché en grand (jaune) ; jamais de numéro de téléphone	M
F-17	Suivi temps réel : approche puis trajet, ETA mis à jour, itinéraire planifié affiché	M
F-18	Alerte de déviation : bannière rouge + vibration + son, actions « Écrire au chauffeur », « Appeler le 117 », « Partager mon trajet »	M
F-19	Partage de trajet : lien public temporaire (position + ETA, sans données personnelles), expiration 2 h après la fin	S
F-20	Urgence : appel direct 117 / 118 (numéros par zone), position et n° de course envoyés au support	M
F-21	Chat in-app avec messages prédéfinis, sans échange de numéros ; historique lié à la course	M
F-22	Annulation après acceptation : frais affichés avant confirmation, selon la grille tarifaire	M

Réf.	Exigence	Prio
F-23	Fin de course : prix final, débit, nouveau solde, notation 1–5 + commentaire, bouton « Contester » (24 h)	M
F-24	Wallet : solde disponible et réservé, recharge MTN MoMo / Airtel Money (USSD push), historique avec référence opérateur	M
F-25	Historique des courses avec reçu téléchargeable (PDF)	S

4.3 Chauffeur (`/d/*`)

Réf.	Exigence	Prio
F-30	Onboarding : téléversement permis, carte grise, assurance, carte de taxi (si taxi), pièce d'identité, photo de profil, photo du véhicule ; statut par pièce (à ajouter / en vérification / validé / refusé avec motif)	M
F-31	Véhicules : plusieurs déclarés, un seul actif, chaque véhicule rattaché à une catégorie	M
F-32	Bascule En ligne / Hors ligne ; en ligne, position envoyée toutes les 3–5 s (10 s à l'arrêt) ; Wake Lock demandé ; bandeau si aucune position depuis 30 s	M
F-33	Offre de course : écran plein, point de prise en charge, distance d'approche, destination, gain estimé, compte à rebours 15 s ; 3 refus consécutifs = pause 2 min	M
F-34	Fiche passager : prénom, initiale, note ; aucun numéro	M
F-35	Navigation : lien « Ouvrir dans Google Maps » vers la prise en charge puis la destination	M
F-36	Arrivée : bouton « Je suis arrivé » actif à moins de 200 m du point de prise en charge ; démarre l'attente gratuite	M
F-37	Saisie du code : 4 chiffres ; erreur affichée ; 3 erreurs verrouillent la course et alertent le support	M

Réf.	Exigence	Prio
F-38	Signaler un détour justifié (route barrée) : journalisé, visible du passager, désactive l'alerte 5 min	S
F-39	Fin de course : bouton actif à moins de 200 m de la destination ; récapitulatif prix / commission / gain net ; notation du passager	M
F-40	Wallet chauffeur : solde, historique, retrait vers MTN MoMo ou Airtel Money (minimum et plafond journalier paramétrables)	M
F-41	Mode dégradé : positions et changements d'état mis en file locale (IndexedDB) et renvoyés au retour du réseau	M

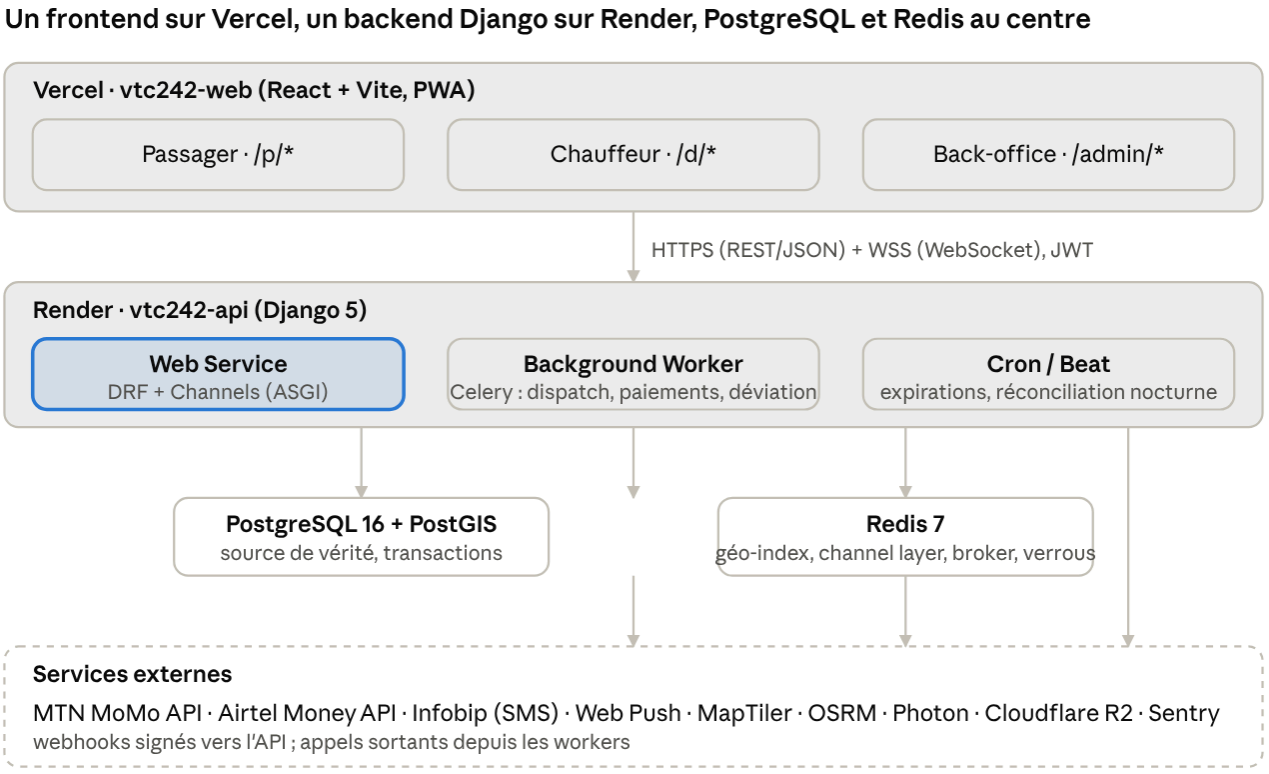
4.4 Back-office (`/admin/*`)

Réf.	Exigence	Prio
F-50	Tableau de bord : courses en cours, chauffeurs en ligne / disponibles, demandes sans chauffeur, CA et commissions du jour, taux d'annulation ; rafraîchi en temps réel	M
F-51	Carte de la flotte en direct avec état par couleur ; clic = fiche course	M
F-52	Journal des alertes (déviation, urgence, codes erronés, détours) avec prise en charge par un opérateur et accès au chat	M
F-53	Chauffeurs : file de validation, aperçu des documents, validation / refus avec motif / suspension, historique	M
F-54	Passagers : recherche, historique, blocage	M
F-55	Grille tarifaire par catégorie et zone : prise en charge, prix/km, prix/min, minimum, attente gratuite, prix/min d'attente, frais d'annulation, commission, majoration ; règles versionnées avec date d'effet ; simulateur de prix	M

Réf.	Exigence	Prio
F-56	Zones : polygones sur carte, numéros d'urgence, seuils de déviation, activation du paiement espèces	M
F-57	Points d'intérêt locaux (ajout, alias, position)	S
F-58	Litiges : liste, trajets prévu / réel côte à côte, remboursement total ou partiel, rejet motivé	M
F-59	Finances : flux wallets, retraits à traiter, import des relevés opérateurs, rapport d'écarts, exports CSV	M
F-60	Rôles admin / finance / support et journal d'audit de chaque action	M

5. Architecture technique et stack

Un monolithe Django modulaire expose une API REST et un WebSocket ; un frontend React unique (Vite) sert les trois interfaces. Le backend et ses workers tournent sur Render, le frontend sur Vercel, la base sur PostgreSQL managé avec PostGIS.



architecture V1 · frontend Vercel, backend Render, données, services externes

Les trois interfaces sont une seule application React ; l'API, le WebSocket et les workers partagent le même code Django et la même base.

Couche	Technologie retenue	Rôle et justification
Backend API	Python 3.12 · Django 5.1 · Django REST Framework · drf-spectacular	ORM, admin, GeoDjango natif ; OpenAPI 3.1 généré
Temps réel	Django Channels 4 (ASGI, Uvicorn) · channel layer Redis	WebSocket dans le même code que l'API, même JWT
Tâches asynchrones	Celery 5 · Celery Beat · broker Redis	Dispatch, webhooks Mobile Money, polling de secours, expirations, réconciliation
Base de données	PostgreSQL 16 + PostGIS 3.4 (Render Postgres ou Neon)	Requêtes spatiales, transactions financières
Cache / géo-index	Redis 7 (Render Key Value ou Upstash) : GEOSEARCH, verrous, channel layer, broker	Positions chauffeurs en mémoire
Frontend	React 18 · Vite · TypeScript · React Router · TanStack Query · Zustand · Tailwind CSS	Stack habituelle de l'équipe ; SPA sans besoin de SEO (tout est derrière connexion)
Cartes	MapLibre GL JS + tuiles MapTiler (plan gratuit puis payant) · OSRM public ou auto-hébergé (routage) · Photon (autocomplétion)	Données OSM correctes sur Brazzaville, pas de clé Google
PWA	vite-plugin-pwa (Workbox) : manifest, service worker, cache des assets, file hors ligne IndexedDB (Dexie)	Installation sur l'écran d'accueil, résilience réseau
Notifications	Web Push (VAPID) via `pywebpush` · SMS Infobip (fallback Twilio)	Web Push non garanti sur iOS : SMS en complément
Fichiers	Cloudflare R2 (compatible S3, sans frais de sortie) via `django-storages`	Documents chauffeurs, photos
Auth	djangorestframework-simplejwt : access 15 min, refresh 30 j rotatif, blacklist	—

Couche	Technologie retenue	Rôle et justification
Hébergement	Render : Web Service (API+WS), Background Worker (Celery), Cron (Beat), Postgres, Key Value · Vercel : frontend	Voir section 11
Observabilité	Sentry (backend + frontend) · Better Stack ou UptimeRobot · logs JSON Render	Alertes erreurs et disponibilité

Pourquoi pas Next.js. L'application est entièrement derrière connexion et n'a pas de contenu à indexer ; le rendu serveur n'apporte rien et complique le hors ligne. Si un site vitrine public est voulu, il se fait en Next.js séparé (`www.vtc242.cg`), le produit reste `app.vtc242.cg` .

Organisation du code (deux dépôts Git)

Dépôt	Structure
`vtc242-api`	`config/` (settings par environnement) · apps Django : `accounts` , `drivers` , `geo` , `rides` , `payments` , `realtime` , `safety` , `backoffice` · `tests/` · `Dockerfile` · `render.yaml`
`vtc242-web`	`src/app` (routes) · `src/features/{passenger,driver,admin}` · `src/shared/{api,ui,map,ws}` · `src/pwa` · tests Vitest + Playwright · `vercel.json`

Chaque app Django expose ses modèles, ses serializers, ses vues et ses tâches Celery ; aucune app n'importe les modèles d'une autre directement, elle passe par des services (`rides/services.py` , `payments/services.py`). La couche `payments.providers` définit une interface `PaymentProvider` (`initiate_collection` , `initiate_disbursement` , `get_status` , `parse_webhook`) avec trois implémentations : `MtnMomoProvider` , `AirtelMoneyProvider` , `SandboxProvider` (tests et démo).

6. Modèle de données

Toutes les tables ont une clé primaire UUID, `created_at` / `updated_at` . Coordonnées en SRID 4326. Montants en `NUMERIC(12,2)` en XAF. Les migrations Django sont la source de vérité ; le DDL ci-dessous fixe les contraintes attendues.

6.1 Types énumérés

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE TYPE user_role          AS ENUM
('passenger','driver','admin','finance','support');
CREATE TYPE vehicle_category   AS ENUM
('taxi','standard','vip','wedding','moving');
CREATE TYPE driver_status      AS ENUM
('pending','approved','suspended','rejected');
CREATE TYPE document_type      AS ENUM
('license','registration','insurance','taxi_card','id_card','vehicle_photo');
CREATE TYPE ride_type          AS ENUM
('immediate','scheduled','shared','rental');   -- scheduled/shared/rental :
V3
CREATE TYPE ride_status        AS ENUM
('requested','dispatching','accepted','arrived','in_progress',

'completed','cancelled_by_passenger','cancelled_by_driver','no_driver');
CREATE TYPE payment_method     AS ENUM ('wallet','cash');
CREATE TYPE mobile_operator    AS ENUM ('mtn','airtel');
CREATE TYPE transaction_type   AS ENUM
('deposit','ride_payment','ride_earning','commission','payout','refund','canc
ellation_fee','adjustment');
CREATE TYPE transaction_status AS ENUM
('pending','completed','failed','expired');
```

6.2 Tables

Table	Rôle	Colonnes clés
`users`	Tous les comptes	`phone_number` UNIQUE, `first_name`, `last_name`, `role`, `photo_url`, `is_active`, `deleted_at`
`otp_codes`	OTP SMS	`phone_number`, `code_hash`, `expires_at`, `attempts`
`push_subscriptions`	Web Push	`user_id`, `endpoint` UNIQUE, `p256dh`, `auth`, `user_agent`
`drivers`	Profil chauffeur (1:1 users)	`status`, `license_number`, `rating_avg`, `rating_count`, `active_vehicle_id`, `is_online`, `is_available`, `current_location` Point, `last_location_at`, `zone_id`, `consecutive_declines`, `paused_until`

Table	Rôle	Colonnes clés
`vehicles`	Véhicules (N:1 drivers)	`brand`, `model`, `color`, `license_plate` UNIQUE, `taxi_number`, `category`, `seats`, `is_verified`
`driver_documents`	Pièces	`driver_id`, `type`, `file_url`, `expires_at`, `status`, `reviewed_by`, `review_note`
`zones`	Périmètres	`name`, `city`, `polygon` MultiPolygon, `emergency_police`, `emergency_fire`, `cash_enabled`, `deviation_threshold_m` (300), `deviation_hits` (3), `is_active`
`points_of_interest`	Lieux locaux	`name`, `aliases text[]`, `location`, `zone_id`
`pricing_rules`	Grille versionnée	`zone_id`, `category`, `base_fare`, `per_km`, `per_minute`, `minimum_fare`, `free_waiting_seconds`, `waiting_per_minute`, `cancellation_fee`, `commission_rate`, `surge_multiplier`, `cash_enabled`, `effective_from`, `effective_to`
`rides`	Courses	voir DDL 6.3
`ride_events`	Journal immuable	`ride_id`, `event_type`, `actor_id`, `payload jsonb`, `occurred_at`
`ride_offers`	Offres de dispatch	`ride_id`, `driver_id`, `offered_at`, `expires_at`, `response` (accepted / declined / timeout)
`ride_positions`	Positions GPS brutes (90 j)	`ride_id`, `driver_id`, `location`, `speed_kmh`, `heading`, `accuracy_m`, `recorded_at`
`ride_ratings`	Notations	`ride_id`, `rater_id`, `ratee_id`, `score` 1–5, `comment`
`chat_messages`	Chat	`ride_id`, `sender_id`, `body`, `sent_at`, `read_at`
`route_deviations`	Alertes	`ride_id`, `recorded_location`, `distance_off_m`, `consecutive_count`, `alert_sent_at`, `acknowledged_by`, `acknowledged_at`

Table	Rôle	Colonnes clés
`emergency_calls`	Urgences	`user_id`, `ride_id`, `number_called`, `location`, `occurred_at`
`trip_shares`	Partage de trajet	`ride_id`, `token` UNIQUE, `expires_at`
`wallets`	1:1 users	`balance`, `reserved_balance`, `currency` ; `CHECK (balance >= 0 AND reserved_balance >= 0 AND reserved_balance <= balance)` (chauffeur : `balance` peut être négatif jusqu'à `zones.cash_debt_limit`)
`wallet_transactions`	Grand livre	`wallet_id`, `ride_id`, `type`, `status`, `amount` signé, `method`, `operator`, `idempotency_key` UNIQUE, `external_reference` UNIQUE, `provider_payload jsonb`, `completed_at`
`payout_requests`	Retraits	`driver_id`, `amount`, `operator`, `phone_number`, `status`, `transaction_id`, `processed_at`, `processed_by`
`disputes`	Litiges	`ride_id`, `opened_by`, `reason`, `status`, `resolution`, `refund_amount`, `resolved_by`, `resolved_at`
`audit_logs`	Actions back-office	`actor_id`, `action`, `target_type`, `target_id`, `diff jsonb`

6.3 DDL de référence — `rides` et `wallets`

```
CREATE TABLE rides (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type                ride_type NOT NULL DEFAULT 'immediate',
  passenger_id        UUID NOT NULL REFERENCES users(id),
  driver_id           UUID REFERENCES drivers(id),
  vehicle_id          UUID REFERENCES vehicles(id),
  category            vehicle_category NOT NULL,
  status              ride_status NOT NULL DEFAULT 'requested',
  payment_method      payment_method NOT NULL DEFAULT 'wallet',
  pricing_rule_id     UUID NOT NULL REFERENCES pricing_rules(id),
  zone_id             UUID NOT NULL REFERENCES zones(id),
  otp_code_hash       TEXT NOT NULL,            -- 4 chiffres, haché ;
```

```
renvoyé en clair au passager seulement
  otp_attempts        SMALLINT NOT NULL DEFAULT 0,
  otp_verified_at     TIMESTAMPTZ,
  pickup_location     GEOMETRY(Point, 4326) NOT NULL,
  dropoff_location    GEOMETRY(Point, 4326) NOT NULL,
  pickup_address      TEXT NOT NULL,
  dropoff_address     TEXT NOT NULL,
  planned_route       GEOMETRY(LineString, 4326),
  planned_distance_m  INTEGER,
  planned_duration_s  INTEGER,
  actual_route        GEOMETRY(LineString, 4326),
  actual_distance_m   INTEGER,
  waiting_seconds     INTEGER NOT NULL DEFAULT 0,
  ride_seconds        INTEGER NOT NULL DEFAULT 0,
  estimated_price_min NUMERIC(12,2) NOT NULL,
  estimated_price_max NUMERIC(12,2) NOT NULL,
  reserved_amount     NUMERIC(12,2) NOT NULL DEFAULT 0,
  final_price         NUMERIC(12,2),
  commission_amount   NUMERIC(12,2),
  driver_earning      NUMERIC(12,2),
  requested_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  accepted_at TIMESTAMPTZ, arrived_at TIMESTAMPTZ, started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ, cancelled_at TIMESTAMPTZ,
  cancel_reason       TEXT,
  dispute_deadline_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_rides_status    ON rides(status);
CREATE INDEX idx_rides_passenger ON rides(passenger_id, requested_at DESC);
CREATE INDEX idx_rides_driver    ON rides(driver_id, requested_at DESC);
CREATE INDEX idx_rides_pickup    ON rides USING GIST(pickup_location);
CREATE UNIQUE INDEX uq_rides_active_passenger ON rides(passenger_id)
  WHERE status IN
('requested','dispatching','accepted','arrived','in_progress');

CREATE TABLE wallets (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID UNIQUE NOT NULL REFERENCES users(id),
  balance          NUMERIC(12,2) NOT NULL DEFAULT 0,
  reserved_balance NUMERIC(12,2) NOT NULL DEFAULT 0,
  currency         CHAR(3) NOT NULL DEFAULT 'XAF',
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (reserved_balance >= 0 AND reserved_balance <= GREATEST(balance, 0))
);
```

Règles d'intégrité. Le solde n'est jamais modifié directement : chaque mouvement est une ligne de `wallet_transactions`, et le solde est mis à jour dans la même transaction SQL sous `SELECT … FOR UPDATE`. Une tâche nocturne recalcule chaque solde depuis les transactions `completed` et alerte en cas d'écart. Une course active par passager est garantie par l'index partiel `uq_rides_active_passenger`.

7. API REST, WebSocket et machine à états

Base `https://api.vtc242.cg/api/v1`, JSON, `Authorization: Bearer <JWT>`, pagination par curseur, erreurs `{ code, message, details }`. Documentation OpenAPI 3.1 générée par drf-spectacular sur `/api/docs`. Les POST financiers et la création de course acceptent un en-tête `Idempotency-Key` (UUID) : la même clé renvoie la même réponse pendant 24 h.

7.1 Endpoints

Méthode et route	Acteur	Comportement
`POST /auth/otp/request`	Tous	Envoie l'OTP SMS ; 429 au-delà de 3 envois / 15 min par numéro
`POST /auth/otp/verify`	Tous	Vérifie, renvoie `access` + `refresh`, crée le compte s'il n'existe pas (rôle passager par défaut)
`POST /auth/token/refresh` · `POST /auth/logout`	Tous	Rotation ; blacklist du refresh
`GET / PATCH /me` · `DELETE /me`	Tous	Profil ; suppression = anonymisation
`POST /me/push-subscriptions`	Tous	Enregistre l'abonnement Web Push
`GET /geo/autocomplete?q=&lat=&lng=`	Passager	Photon + points d'intérêt de la zone
`GET /geo/zones/resolve?lat=&lng=`	Tous	Zone couvrant un point (404 hors zone)
`POST /rides/estimate`	Passager	`{pickup, dropoff}` → par catégorie : fourchette, distance, durée, ETA d'approche, `pricing_rule_id`, `cash_enabled`

Méthode et route	Acteur	Comportement
`POST /rides`	Passager	Crée la course, réserve `estimated_price_max`, lance le dispatch. 402 solde insuffisant, 409 course déjà active, 422 hors zone
`GET /rides/{id}`	Passager, chauffeur	`otp_code` renvoyé au passager seulement ; jamais de numéro de téléphone de l'autre partie
`GET /rides?status=`	Passager, chauffeur	Historique
`POST /rides/{id}/cancel`	Passager, chauffeur	Motif ; frais selon règle et statut ; libère la réservation
`POST /rides/{id}/accept`	Chauffeur	409 si déjà attribuée ou offre expirée
`POST /rides/{id}/decline`	Chauffeur	Refus explicite ; passe au suivant
`POST /rides/{id}/arrive`	Chauffeur	422 si à plus de 200 m du point de prise en charge
`POST /rides/{id}/start`	Chauffeur	`{otp_code}` ; 400 invalide, 423 verrouillée après 3 échecs
`POST /rides/{id}/detour`	Chauffeur	Signale un détour justifié
`POST /rides/{id}/complete`	Chauffeur	422 si à plus de 200 m de la destination (sauf validation passager) ; calcule et débite
`POST /rides/{id}/confirm-arrival`	Passager	Autorise la clôture hors rayon
`POST /rides/{id}/rate`	Passager, chauffeur	1–5 + commentaire, une fois
`POST /rides/{id}/dispute`	Passager	Dans les 24 h
`GET / POST /rides/{id}/messages`	Passager, chauffeur	Miroir REST du chat
`POST /rides/{id}/share`	Passager	Crée un lien public ; `GET /share/{token}` sans authentification

Méthode et route	Acteur	Comportement
`POST /rides/{id}/emergency`	Passager, chauffeur	Journalise l'appel et alerte le support
`GET /wallet` · `GET /wallet/transactions`	Passager, chauffeur	Solde disponible / réservé, historique
`POST /wallet/deposit`	Passager, chauffeur	`{amount, operator, phone_number}` → 202 + transaction `pending`
`POST /wallet/payout`	Chauffeur	Retrait ; 422 sous le minimum ou au-delà du plafond journalier
`POST /payments/webhooks/mtn` · `/airtel`	Opérateurs	Signature vérifiée, 200 immédiat, traitement Celery
`PATCH /drivers/me/status`	Chauffeur	`{is_online}`
`POST /drivers/me/location`	Chauffeur	Fallback REST du WebSocket (mode dégradé, batch de positions)
`GET/POST /drivers/me/vehicles` · `POST /drivers/me/documents`	Chauffeur	Véhicules ; upload via URL pré-signée R2
`/admin/*`	Admin, finance, support	Un endpoint par écran de la section 4.4, protégé par rôle et journalisé

7.2 Machine à états d'une course

```
requested ──▶ dispatching ──▶ accepted ──▶ arrived ──▶ in_progress ──▶ completed
    │              │              │            │
    │              └─▶ no_driver  └────────────┴──▶ cancelled_by_passenger / cancelled_by_driver
    └─▶ cancelled_by_passenger
```

Chaque transition passe par `rides/services.py` (jamais par un `ride.status = …` direct), écrit une ligne dans `ride_events` et émet `ride.status_updated`. Une transition interdite renvoie 409.

7.3 WebSocket

URL `wss://api.vtc242.cg/ws/?token=<access JWT>`. Un canal par utilisateur ; le serveur abonne le client aux groupes `user.<id>` et `ride.<id>`. Ping toutes les 20 s, fermeture après 60 s de silence ; le client se reconnecte avec backoff (1, 2, 4, 8 s) et rejoue via REST ce qu'il a manqué (`GET /rides/{id}`). Toute trame : `{ "event": "…", "ts": <ms>, "data": {…} }`.

Événement	Sens	Données
`driver.location`	Chauffeur → serveur	`lat, lng, heading, speed_kmh, accuracy_m, recorded_at, ride_id?` ; ignoré si `accuracy_m > 100`
`ride.offer`	Serveur → chauffeur	`ride_id, category, pickup{address,lat,lng,distance_m}, dropoff{address}, estimated_earning, expires_at`
`ride.offer_response`	Chauffeur → serveur	`ride_id, accepted`
`ride.status_updated`	Serveur → passager, chauffeur	`ride_id, status`, `driver{first_name, photo_url, rating, vehicle{brand, model, color, license_plate, taxi_number}}` (passager), `passenger{first_name, rating}` (chauffeur), `otp_code` (passager uniquement)
`ride.driver_position`	Serveur → passager	`ride_id, lat, lng, heading, eta_s` (au plus toutes les 3 s)
`ride.deviation_alert` · `ride.deviation_cleared`	Serveur → passager, back-office	`ride_id, distance_off_m, emergency_numbers`
`chat.message`	Bidirectionnel	`ride_id, message_id, sender_role, body, sent_at`
`ride.cancelled`	Serveur → concerné	`ride_id, by, reason, fee`
`wallet.updated`	Serveur → utilisateur	`balance, reserved_balance`
`admin.dashboard`	Serveur → back-office	KPI + alertes, toutes les 5 s

Aucun événement ne transporte de numéro de téléphone.

7.4 Dispatch (tâche Celery `rides.dispatch`)

1. `GEOSEARCH drivers:geo:<category> FROMLONLAT … BYRADIUS 2000 m ASC COUNT 10` ; si vide, 4 km puis 6 km.
2. Filtre : chauffeur disponible, non refusant sur cette course, non en pause, note ⩾ seuil de la zone.
3. Score = distance pondérée par le temps d'approche OSRM et la note ; le meilleur reçoit `ride.offer` avec verrou `SET NX EX 15 ride:lock:<ride_id>`.
4. Sans réponse en 15 s : chauffeur suivant. Après 5 offres ou 2 min : `no_driver`, fonds libérés, message au passager.

7.5 Détection de déviation (tâche `safety.check_position`)

Au `start`, l'itinéraire OSRM est stocké dans `rides.planned_route` et échantillonné tous les 50 m dans Redis (`ride:route:<id>`, GEOADD). Pour chaque position : GEOSEARCH dans un rayon de `deviation_threshold_m` → dans le couloir, compteur remis à 0 ; sinon confirmation PostGIS `ST_Distance` ; si toujours hors couloir, `HINCRBY` ; à `deviation_hits` positions consécutives → alerte (WebSocket, push, ligne `route_deviations`). Une alerte par tranche de 2 min ; levée automatique au retour dans le couloir ; suspendue 5 min après un détour signalé.

8. Paiements Mobile Money et wallet

Le wallet VTC 242 est l'unité de compte interne ; MTN MoMo et Airtel Money servent uniquement à l'alimenter (collection) et à le vider (disbursement). Une course en mode wallet suit : réservation de `estimated_price_max` à la commande → débit du prix final à la clôture → libération du reliquat. En mode espèces, aucun mouvement passager ; la commission est portée au débit du wallet chauffeur, dont le solde peut devenir négatif jusqu'à un plafond de zone ; au-delà, le chauffeur ne peut plus passer en ligne.

8.1 Recharge (collection)

1. `POST /wallet/deposit` crée une `wallet_transactions` `pending` avec `idempotency_key` et répond 202.
2. Le worker appelle `provider.initiate_collection(amount, phone, idempotency_key)` ; l'opérateur envoie l'USSD push ; l'utilisateur saisit son PIN.
3. Le webhook signé arrive sur `/payments/webhooks/<operator>` : réponse 200 immédiate, traitement en tâche Celery qui vérifie le statut auprès de l'API opérateur avant de passer la transaction en `completed` et de créditer le wallet.
4. Sans webhook après 90 s, polling `get_status` toutes les 30 s ; après 15 min, `expired`.

5. `wallet.updated` en WebSocket + push « Rechargement réussi ».

8.2 Retrait chauffeur (disbursement)

`POST /wallet/payout` → vérification minimum / plafond journalier → transaction `payout pending` et débit immédiat → `initiate_disbursement` → confirmation par webhook ou polling → `completed` ; en cas d'échec confirmé, recrédit automatique et notification. Un rôle finance peut mettre les retraits en validation manuelle au-dessus d'un seuil.

8.3 Connecteurs

	MTN MoMo (Collection / Disbursement API)	Airtel Money (Open API)
Environnement	Sandbox MoMo Developer Portal puis production, `X-Target-Environment: mtncongo`	Sandbox openapi.airtel.africa puis production, `X-Country: CG`, `X-Currency: XAF`
Authentification	API User + API Key → token OAuth2, renouvelé avant expiration	`client_id` / `client_secret` → token OAuth2
Collection	`POST /collection/v1_0/requesttopay`, `X-Reference-Id` = `idempotency_key`	`POST /merchant/v1/payments/`, `reference` = `idempotency_key`
Statut	`GET /collection/v1_0/requesttopay/{referenceId}`	`GET /standard/v1/payments/{id}`
Disbursement	`POST /disbursement/v1_0/transfer`	`POST /standard/v1/disbursements/`
Webhook	Callback URL déclarée à la création de l'API User	Callback URL configurée dans le portail

Ces éléments viennent de la documentation publique ; identifiants de production, URLs de callback et signature exacte des webhooks sont fournis par les opérateurs lors de l'onboarding marchand (plusieurs semaines). `SandboxProvider` simule les deux flux (succès, refus, timeout) pour développer et recetter sans dépendre de ce calendrier.

8.4 Règles financières

- Idempotence stricte : une clé par tentative ; webhook ou réponse dupliqués ignorés (`external_reference` UNIQUE).
- Prix final = `max(minimum_fare, (base_fare + km × per_km + min × per_minute + attente payante × waiting_per_minute) × surge_multiplier)`, arrondi aux 100 F supérieurs.
- Commission = `final_price × commission_rate`, arrondie au franc, transaction distincte `commission`.
- Si le prix final dépasse le montant réservé + le solde disponible, le débit est plafonné et la différence est signalée au back-office (cas rare : attente prolongée).

- Annulation après acceptation : `cancellation_fee` débité au passager, dont 50 % (paramétrable) crédités au chauffeur.
- Réconciliation quotidienne : import CSV des relevés opérateurs, rapprochement sur `external_reference`, rapport d'écarts envoyé au rôle finance.

9. Sécurité, confidentialité, conformité

Domaine	Exigence
Transport	HTTPS / WSS partout, HSTS, en-têtes `django-csp`, `SECURE_*` activés ; CORS limité à `app.vtc242.cg` et aux previews Vercel du projet
Authentification	JWT access 15 min, refresh 30 j rotatif avec blacklist ; un appareil actif par chauffeur ; OTP SMS et code de départ stockés hachés (argon2)
Limitation de débit	`django-ratelimit` : 3 OTP / 15 min / numéro, 10 tentatives / min sur `/start`, 60 req / min par utilisateur sur l'API
Webhooks	Vérification HMAC ou signature opérateur, liste blanche d'IP quand fournie, réponse 200 puis traitement asynchrone, charge brute conservée 1 an dans `provider_payload`
Données personnelles	Minimisation : prénom + initiale exposés à l'autre partie, jamais le numéro ; chiffrement des documents au repos (R2 SSE) ; positions brutes 90 j, transactions 10 ans, chat 1 an ; export et suppression de compte ; conformité à la loi congolaise n° 29-2019 sur la protection des données à valider avec un conseil local
Secrets	Variables d'environnement Render / Vercel uniquement, jamais dans Git ; rotation des clés opérateurs documentée
Fichiers	Upload via URL pré-signée, types autorisés (jpg, png, pdf), 10 Mo max, scan antivirus optionnel (ClamAV) en V2
Back-office	2FA TOTP obligatoire pour `admin` et `finance` ; journal d'audit immuable ; sessions 8 h

Domaine	Exigence
Frontend	Aucun secret dans le bundle ; tokens en mémoire + refresh en cookie `HttpOnly` `SameSite=None; Secure` ; protection XSS via React, CSP stricte
Dépendances	Dependabot activé ; `pip-audit` et `npm audit` en CI, échec sur vulnérabilité critique

10. Exigences non fonctionnelles, responsive et PWA

Réf.	Domaine	Exigence
N-01	Responsive	Points de rupture : 360 px (téléphone), 768 px (tablette), 1024 px et 1440 px (desktop). Passager et chauffeur : mobile-first, une colonne, boutons ⩾ 48 px, carte plein écran avec panneau glissant. Back-office : desktop-first, barre latérale repliable sous 1024 px, tableaux défilant horizontalement, KPI en grille auto-adaptative. Aucun défilement horizontal de page à aucune largeur.
N-02	Navigateurs	Chrome et Samsung Internet Android 8+ (2 dernières versions), Safari iOS 16.4+, Chrome / Edge / Firefox desktop. Testé sur un Android d'entrée de gamme (2 Go RAM)
N-03	PWA	Manifest (nom, icônes, couleur de thème, `display: standalone`), service worker Workbox : assets en cache, données réseau-d'abord, page hors ligne. Invite d'installation après la première course terminée. Score Lighthouse PWA « installable »
N-04	Performance	Bundle initial < 500 Ko compressé (cartes chargées à la demande) ; Lighthouse Performance ⩾ 80 sur mobile simulé 3G ; API p95 ⩽ 150 ms côté serveur (latence Congo ↔ Europe 120–250 ms exclue) ; position chauffeur diffusée au passager en ⩽ 1 s
N-05	Réseau faible	Payloads compacts, gzip / brotli, images WebP, tuiles vectorielles en cache ; file hors ligne chauffeur (IndexedDB) ; cible < 30 Mo / jour pour un chauffeur à plein temps

Réf.	Domaine	Exigence
N-06	Charge V1	300 chauffeurs en ligne (≈ 100 positions / s), 3 000 passagers actifs, 5 000 courses / jour, marge ×5 sans changement d'architecture
N-07	Disponibilité	99,5 % mensuel ; maintenance annoncée la nuit ; page de statut publique
N-08	Sauvegardes	Postgres : sauvegardes quotidiennes Render + `pg_dump` hebdomadaire copié sur R2 ; RPO ⩽ 24 h en V1 (⩽ 5 min avec archivage WAL en V2) ; restauration testée avant la mise en production puis trimestriellement ; RTO ⩽ 2 h
N-09	Accessibilité	Contraste AA, navigation clavier sur le back-office, libellés ARIA sur les boutons d'action, tailles de police ⩾ 16 px sur mobile, respect de `prefers-reduced-motion`
N-10	Charte graphique	Fond blanc, texte `#111318` ; vert `#1E9B45` actions principales ; jaune `#F2C21B` code de départ et compte à rebours uniquement ; rouge `#C8202C` urgence et alertes uniquement ; anthracite `#24272C` back-office et marketing ; police Manrope (400 / 600 / 800) ; conforme au prototype livré
N-11	Internationalisation	Textes externalisés (`react-i18next`) dès la V1, français seul activé ; formats XAF `12 500 F`
N-12	Observabilité	Sentry (erreurs, performance), logs JSON, métriques métier exposées sur `/metrics` (courses par statut, chauffeurs en ligne par zone, durée de matching, paiements par opérateur et statut) ; alertes : 5xx > 1 %, aucun chauffeur en ligne sur une zone active, webhooks en échec, écart de réconciliation
N-13	Qualité de code	Python : `ruff` , `mypy` strict sur `payments` et `rides` , couverture ⩾ 80 % sur ces deux apps ; TypeScript strict, `eslint` , `prettier` ; revue obligatoire, branche `main` protégée
N-14	Documentation	OpenAPI générée, README d'installation locale (Docker Compose : Postgres/PostGIS, Redis, API, worker, web), guide d'exploitation, runbook d'incident

11. Hébergement Render / Vercel, environnements, CI/CD

11.1 Services Render (`render.yaml` dans `vtc242-api`)

Service	Type	Commande	Plan indicatif
`vtc242-api`	Web Service (Docker)	`uvicorn config.asgi:application --host 0.0.0.0 --port $PORT --workers 2`	Standard (2 Go), autoscaling 1–3 instances
`vtc242-worker`	Background Worker	`celery -A config worker -l info --concurrency 4`	Starter puis Standard
`vtc242-beat`	Background Worker	`celery -A config beat -l info` (une seule instance)	Starter
`vtc242-db`	PostgreSQL 16	extension PostGIS activée ; sauvegardes quotidiennes	Basic 1 Go → Pro
`vtc242-redis`	Key Value	politique `noeviction`, persistance activée	Starter
Pré-déploiement	`preDeployCommand`	`python manage.py migrate --noinput && python manage.py collectstatic --noinput`	—

Région Render : Frankfurt (la plus proche des câbles WACS / SAT-3 vers Brazzaville). Le WebSocket est supporté nativement par Render ; l'API et le WS partagent le même service pour garder un seul JWT et une seule origine.

11.2 Vercel (`vtc242-web`)

- Build `vite build`, sortie `dist/`, `vercel.json` avec `rewrites` de toutes les routes vers `/index.html` (SPA) et en-têtes de cache longs sur `/assets/*`.
- Domaines : `app.vtc242.cg` (production), `staging.vtc242.cg` (branche `staging`), previews automatiques par pull request.
- Variables : `VITE_API_URL`, `VITE_WS_URL`, `VITE_MAPTILER_KEY`, `VITE_VAPID_PUBLIC_KEY`, `VITE_SENTRY_DSN`.

11.3 Environnements

Environnement	Backend	Frontend	Données	Paiements
Local	Docker Compose (Postgres/PostGIS, Redis, API, worker)	`vite dev`	jeu de données de démo (`manage.py seed_demo`)	`SandboxProvider`
Staging	Render, branche `staging` , services Starter	Vercel `staging.vtc242.cg`	copie anonymisée	Sandbox opérateurs
Production	Render, branche `main` , promotion manuelle après validation staging	Vercel `app.vtc242.cg`	production, sauvegardes	Production opérateurs

11.4 Variables d'environnement backend (extrait)

`DJANGO_SETTINGS_MODULE` , `SECRET_KEY` , `DATABASE_URL` , `REDIS_URL` , `ALLOWED_HOSTS` , `CORS_ALLOWED_ORIGINS` , `FRONTEND_URL` , `R2_ACCOUNT_ID` / `R2_ACCESS_KEY` / `R2_SECRET_KEY` / `R2_BUCKET` , `INFOBIP_API_KEY` , `VAPID_PRIVATE_KEY` , `MTN_API_USER` / `MTN_API_KEY` / `MTN_SUBSCRIPTION_KEY` / `MTN_ENV` , `AIRTEL_CLIENT_ID` / `AIRTEL_CLIENT_SECRET` / `AIRTEL_ENV` , `OSRM_URL` , `PHOTON_URL` , `SENTRY_DSN` , `PAYMENT_PROVIDER_MODE` (`sandbox` / `live`).

11.5 CI/CD (GitHub Actions)

Déclencheur	Backend	Frontend
Pull request	`ruff` , `mypy` , `pytest` avec Postgres/PostGIS + Redis en services, `pip-audit` , build Docker	`eslint` , `tsc` , `vitest` , `npm audit` , build ; preview Vercel automatique
Push `staging`	Déploiement Render staging (auto-deploy)	Déploiement Vercel staging
Push `main`	Déploiement Render production (auto-deploy) + tag de version	Déploiement Vercel production
Nocturne	Tests Playwright de bout en bout sur staging (parcours de la section 3) ; sauvegarde `pg_dump` vers R2	—

11.6 Coût mensuel indicatif au lancement

Render ≈ 60 à 90 $ (web Standard, worker, beat, Postgres Basic, Key Value) ; Vercel Pro 20 $; MapTiler 0 à 25 $; Cloudflare R2 < 5 $; Sentry 0 à 26 $; SMS Infobip selon volume (≈ 0,04 € / SMS). Total hors SMS : 100 à 170 $ / mois.

12. Organisation, planning et livrables

12.1 Équipe

Rôle	Charge	Responsabilités
Chef de projet / product owner	0,3 ETP	Backlog, priorisation, recette, relation client
Développeur backend Django	1 ETP	API, temps réel, dispatch, paiements, tâches
Développeur frontend React	1 ETP	Passager, chauffeur, PWA, cartes
Développeur frontend (back-office)	0,5 ETP	Écrans admin, exports
UI designer	0,2 ETP (phase 1)	Maquettes haute fidélité à partir du prototype, design system
DevOps (partagé)	0,1 ETP	Render, Vercel, CI/CD, supervision
Référent terrain Brazzaville	Client	Chauffeurs pilotes, relation MTN / Airtel, tests réels

12.2 Sprints (2 semaines) — V1 en 12 sprints, soit 24 semaines

Sprint	Livrable vérifiable
0	Cadrage : validation de ce document, dossiers marchands MTN / Airtel déposés, comptes Render / Vercel / R2 / Infobip / MapTiler créés, dépôts et CI en place, environnements local et staging opérationnels
1	Maquettes haute fidélité des trois interfaces validées ; modèle de données migré ; auth OTP + JWT ; gestion des profils
2	Zones, grille tarifaire, estimation de prix (`/rides/estimate`) avec OSRM ; écran d'accueil et recherche d'adresse passager

Sprint	Livrable vérifiable
3	Création de course, réservation de fonds (wallet en `SandboxProvider`), machine à états, `ride_events` ; écrans commande et recherche de chauffeur
4	Géo-index Redis, WebSocket, envoi de position chauffeur, dispatch avec offres 15 s ; écrans En ligne et Offre côté chauffeur
5	Acceptation, approche, arrivée, code de départ, verrouillage ; fiche chauffeur et code côté passager ; suivi temps réel
6	Course en cours, itinéraire planifié, détection de déviation, alertes, détour ; chat ; urgence ; partage de trajet
7	Clôture, prix final, commission, notation, annulation avec frais, litiges côté passager ; historique
8	Onboarding chauffeur (documents, véhicules), wallet chauffeur, retraits ; mode dégradé hors ligne ; Web Push + SMS
9	Back-office : supervision temps réel, carte, alertes, chauffeurs, passagers
10	Back-office : tarifs et simulateur, zones, litiges, finances, exports, rôles et audit ; PWA installable ; Lighthouse
11	Connecteurs MTN et Airtel en sandbox opérateur puis production ; réconciliation ; tests de charge (k6) ; correctifs
12	Pilote terrain à Brazzaville (20–30 chauffeurs, 2 semaines), calibrage des seuils, corrections ; mise en production ; formation admin ; transfert

Le chemin critique est l'accès de production Mobile Money. S'il glisse, le pilote démarre avec `SandboxProvider` et le paiement en espèces, et le sprint 11 est décalé sans bloquer le reste.

12.3 Livrables

- Dépôts `vtc242-api` et `vtc242-web` avec historique Git, licence cédée au client.

- Documentation : OpenAPI, README d'installation locale, guide d'exploitation Render / Vercel, runbook d'incident, guide utilisateur du back-office.
- Application en production sur `app.vtc242.cg` et `api.vtc242.cg`, comptes cloud au nom du client.
- Maquettes et design system (Figma).
- Rapport de tests (unitaires, intégration, bout en bout, charge, pilote).
- Garantie corrective de 3 mois après mise en production, puis contrat de maintenance optionnel.

12.4 Définition de « terminé » pour chaque ticket

Code revu et fusionné sur `main` · tests unitaires écrits et verts · exigence F-xx référencée dans la PR · déployé sur staging · vérifié sur un téléphone Android réel et sur desktop · documentation OpenAPI à jour si l'API change.

13. Critères d'acceptation et tests

La recette de la V1 est prononcée quand les scénarios ci-dessous passent sur staging avec `SandboxProvider`, puis en production avec un paiement réel de 500 F par opérateur.

Réf.	Scénario	Résultat attendu
A-01	Inscription par OTP sur un Android d'entrée de gamme en 3G	Compte créé en moins de 60 s ; 3 mauvais codes bloquent 15 min
A-02	Parcours nominal complet (section 3) avec 2 téléphones	Chaque étape reflétée sur l'autre appareil en moins de 2 s ; prix final = formule 8.4 ; wallets passager et chauffeur cohérents avec `wallet_transactions`
A-03	Commande avec solde insuffisant	402 et message clair ; aucune réservation créée
A-04	Aucun chauffeur ne répond à 5 offres	`no_driver` en moins de 2 min 30, fonds libérés, passager informé
A-05	Code de départ faux 3 fois	Course verrouillée (423), alerte visible au back-office en moins de 10 s
A-06	Chauffeur quitte l'itinéraire de 400 m pendant 15 s (simulateur GPS)	Alerte passager et back-office ; levée au retour dans le couloir ; aucune alerte lors d'un détour signalé

Réf.	Scénario	Résultat attendu
A-07	Coupure réseau côté chauffeur pendant 60 s en course	Positions rejouées à la reconnexion ; bandeau « position perdue » côté passager après 30 s
A-08	Annulation passager après acceptation	Frais débités, part chauffeur créditée, événement journalisé
A-09	Recharge MTN et Airtel avec webhook, puis sans webhook (polling), puis timeout	`completed` / `completed` / `expired` ; aucun double crédit sur webhook rejoué
A-10	Retrait chauffeur avec échec opérateur simulé	Recrédit automatique et notification
A-11	Validation d'un chauffeur au back-office	SMS envoyé, chauffeur peut passer en ligne ; action dans `audit_logs`
A-12	Modification d'un tarif avec date d'effet	Estimations conformes avant / après la date ; courses en cours conservent leur règle
A-13	Litige remboursé à 50 %	Wallet passager crédité, transaction `refund`, litige `resolved`
A-14	Responsive	Aucune page ne défile horizontalement à 360, 768, 1024 et 1440 px ; captures Playwright archivées
A-15	PWA	Installation depuis Chrome Android ; ouverture hors ligne affiche l'écran dédié ; Lighthouse « installable »
A-16	Charge k6 sur staging : 300 chauffeurs envoyant une position toutes les 4 s, 50 commandes / min pendant 15 min	API p95 ⩽ 150 ms serveur, 0 erreur 5xx, matching < 20 s
A-17	Suppression de compte	Données personnelles anonymisées, transactions conservées, connexion impossible
A-18	Sécurité	`GET /rides/{id}` d'une course d'autrui → 404 ; aucun numéro de téléphone dans les réponses ni les trames WebSocket (test automatisé)

Outils. Backend : `pytest`, `pytest-django`, `factory_boy`, `channels` testing, Postgres/PostGIS de test. Frontend : `vitest` + Testing Library, Playwright pour les parcours (mobile et desktop), Lighthouse CI. Charge : k6. Simulateur GPS : script rejouant des traces GPX de Brazzaville via `POST /drivers/me/location`.

14. Risques, hypothèses et questions ouvertes

Risque	Impact	Mitigation
Délai d'onboarding MTN / Airtel (plusieurs semaines à plusieurs mois)	Élevé	Dossiers déposés au sprint 0 ; `SandboxProvider` ; paiement espèces de repli ; sprint 11 découplé
GPS en arrière-plan impossible dans un navigateur mobile	Élevé	Wake Lock, bandeau d'alerte, consigne écran allumé aux chauffeurs pilotes ; V2 native prioritaire pour le chauffeur
Web Push indisponible sur iOS hors PWA installée	Moyen	SMS pour chauffeur trouvé et code de départ ; invite d'installation
Qualité GPS des téléphones d'entrée de gamme	Moyen	Filtre `accuracy_m`, lissage, seuils calibrés pendant le pilote
Données OSM incomplètes sur certains quartiers	Moyen	Points d'intérêt locaux, saisie libre, contributions OSM pendant le pilote
Cadre réglementaire (taxis conventionnés, statut des chauffeurs, données personnelles)	Élevé	Avis juridique local dès le sprint 0
Fraude (fausses courses, collusion passager-chauffeur)	Moyen	Fonds réservés avant départ, code de départ, journal immuable, alertes back-office sur schémas anormaux
Latence Congo ↔ Europe	Faible	Objectifs mesurés côté serveur ; payloads compacts ; WebSocket plutôt que polling
Coût SMS	Faible	OTP uniquement à la connexion (sessions 30 j) ; SMS critiques limités à 2 par course

Hypothèses

- Le client dispose d'une entité juridique au Congo pour contracter avec MTN et Airtel et détenir les comptes cloud.

- Le client fournit les numéros d'urgence par ville, la grille tarifaire initiale et 20 à 30 chauffeurs pilotes équipés d'Android 8+.
- Les API opérateurs correspondent à leur documentation publique ; tout écart constaté fait l'objet d'un avenant technique sans impact sur le reste du périmètre.
- Le prototype livré en septembre 2026 est la référence UX ; les maquettes du sprint 1 le précisent sans en changer les parcours.

Questions à trancher avant le sprint 1

- [ ] Le paiement en espèces est-il activé au lancement, et pour quelles catégories ?
- [ ] Taux de commission par catégorie ; modèle pour les taxis conventionnés (commission ou abonnement mensuel) ?
- [ ] Politique d'annulation : délai gratuit, montant des frais, part reversée au chauffeur ?
- [ ] Minimum et plafond journalier des retraits chauffeurs ; validation manuelle au-dessus de quel montant ?
- [ ] Chauffeurs uniquement indépendants en V1, ou exploitants gérant plusieurs chauffeurs ?
- [ ] Nom de domaine définitif (`vtc242.cg` ou autre) et propriétaire des comptes Render, Vercel, Infobip, MapTiler ?
- [ ] Faut-il un site vitrine public (Next.js) en plus de l'application ?